

NANYANG TECHNOLOGICAL UNIVERSITY

SC4051: Distributed System

Distributed Banking System

Team S.E.E.

Full Name	Matriculation Number	Percentage of Contribution
Eben Lim	U2423065E	100%
Enzio Dwiputra Sujandi	U2423055G	100%
Sean Winston Susanto	U2423834G	100%

2025/2026

SEMESTER 2

COLLEGE OF COMPUTING AND DATA SCIENCE

Table of Contents

1. Introduction..... 3
1.1. Environment..... 3
1.2. Assumption..... 3
2. System Overview..... 3
2.1. Architecture Design..... 3
2.1.1. Server..... 4
2.1.2. Client..... 5
3. Message Protocol and Data Structures..... 6
3.1. Marshalling..... 6
3.2. Message Structure..... 6
4. Services..... 7
4.1. Service 1: Open New Account..... 7
4.2. Service 2: Close Existing Account..... 7
4.3. Service 3: Deposit/Withdraw Money..... 7
4.4. Service 4: Transfer Money..... 7
4.5. Service 5: View Account..... 8
4.6. Service 6: Monitor Update..... 8
4.7. Service 7: Change Password..... 8
5. Fault Tolerance..... 8
5.1. Timeout Retransmission..... 8
5.2. Filter Duplicate Requests..... 8
6. Invocation Semantics..... 9
6.1. At-Least-Once..... 9
6.2. At-Most-Once..... 9
6.3. Testing..... 9
6.3.1. Idempotent..... 9
6.3.2. Non-Idempotent..... 9
7. Conclusion..... 11

1. Introduction

This project applies the fundamental concepts of interprocess communication and remote invocation through constructing a client-server system, which uses UDP as the transport protocol, to simulate a simple distributed banking system.

1.1. Environment

Our system is developed and tested under the UNIX/MacOS systems using Java as the client and C++ as the server application

1.2. Assumption

The following is the list of assumptions defined for this application:

- Assume that the client application knows the complete server address information before the communication begins
- Assume that both the clients and the server are aware of the same service arguments parameter type and sequence ordering. In the source code, we define this as the ***ServiceSchema***
- Assume that the clients service requests arrival are well separated in time so that there are only one service request at one time, so there are no simultaneous requests within the server process
- No persistent record or file system is used in this system, so the input record within the application only persists during the server runtime on the server memory.
- Clients are not able to issue any new requests while they are in the monitoring session
- Money is represented internally as a long integer (8 bytes) in the units of cents. The normal representation in dollar is used only in the UI

2. System Overview

2.1. Architecture Design

The distributed banking system uses User Datagram Protocol (UDP) as a communication protocol between client and server. It uses a socket as the communication entry between the clients and server. Our system consists of three main layers, that is **Socket Layer, Communication Layer, and Application Layer.**

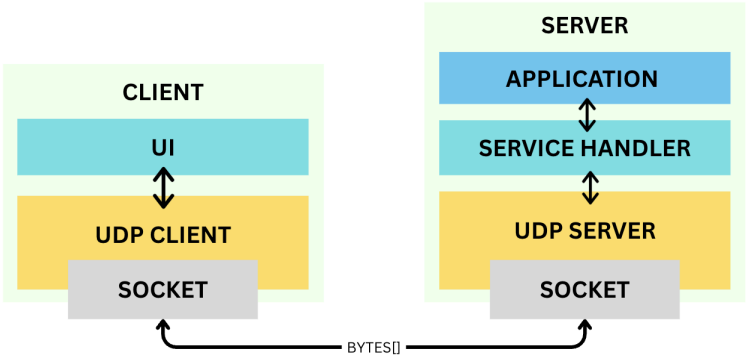


Fig 1. Architecture Design

- **The Socket Layer** is the layer that holds the built-in socket module of the client and server application. It is primarily used only to send and receive the UDP packets from client/server
- **The Communication Layer** is the logical layer that defines the configuration, such as the invocation semantics, max retrial, and packet loss error simulation rate, as well as the logical flow of the communication. This includes the request/response packet building, marshaling/unmarshaling sequences based on defined service schema and callback management.
- **The Application Layer** defines the core logic of the banking application, including account, transaction, and balance management. For client, this includes the UI state management, while for server this defines the Account as the data sublayer, and AccountManager & Service Handler as the control sublayer.

2.1.1. Server

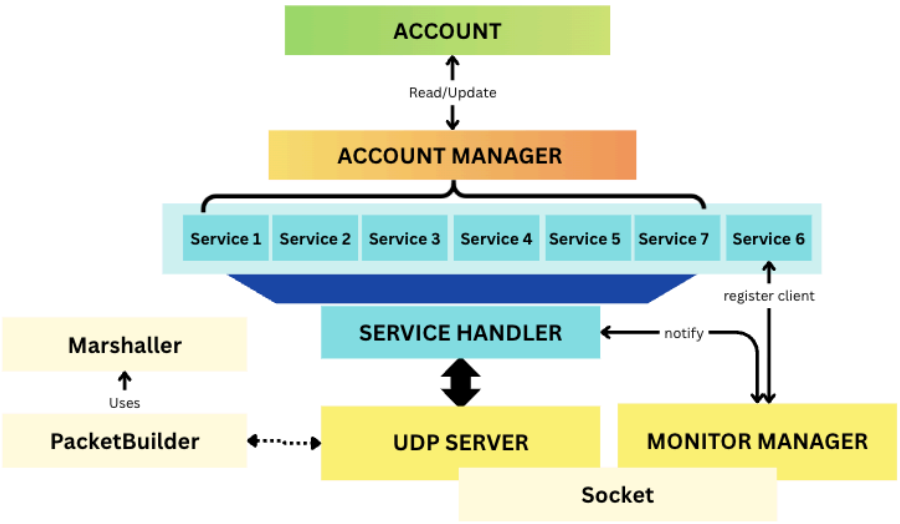


Fig 2. Server Architecture

At the communication level, our server contains UDP server and monitor manager, both of which hold a PacketBuilder that is a utility class used to build and parse the complete request/response messages using the predefined message protocol and service schema. It uses the Marshaller utility to perform the element-wise marshaling/unmarshaling operation.

When the server receives a request message, it marshals the entire packets and extracts the headers and payload to form a *Request* object. This will be passed on to the ServiceHandler, which will process the header information, especially the ServiceID and direct it to the respective services to be further executed by the AccountManager (application level). Upon operation completion, the output results (output and status code) will be passed back to *ServiceHandler,* which will handle the construction of *Response* objects and Notification messages, which will be passed to *UDPServer* and *MonitorManager,* respectively.

Both UDPServer and MonitorManager will then build the raw-byte messages and invoke the socket operation to send the packets to their respective clients. (UDPServer sends to the client caller, while Monitor Manager sends it to Active Monitoring Client).

2.1.2. Client

Similar to the server, the UDP Client also has PacketUtils and Marshaller to build the request messages. The only difference is that the flow begins from the UI side, where the user interacts with the MainUIHandle, which will direct users to the different Service UI Handles to interact with the system. The user input will be passed to the UDPClient which will then build the request messages, send messages to the server. Any responses from the server as a result will be received by the socket and parsed by UDPClient before being passed to the respective UI handler to be processed and shown to the user.

Additionally, the client-side handlers are also responsible for validating user input, formatting service-specific arguments, and invoking the appropriate service requests. Upon receiving a de-serialized response, the client parses the status code and payload to present meaningful feedback to the user.

The *UDPClient* also implements a timeout and retransmission mechanism to handle unreliable communication over UDP. If a response is not received within a specified time, the client resends the request using the same request ID. This ensures reliability while allowing the server to detect and filter duplicate requests.

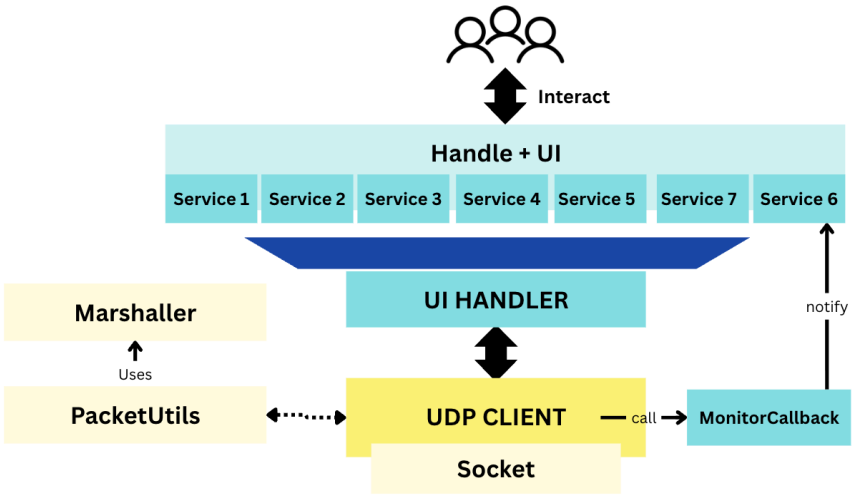


Fig 3. Client Architecture

3. Message Protocol and Data Structures

3.1. Marshalling

Since we send data as raw bytes between server and client, the bytes need to be unmarshalled which will turn bytes to the intended data type. Our application supports common data types, such as 4-byte integer, 8-byte long, 4-byte float, variable-length string and single byte char (UTF-8). Specifically, for integer and float, whereby one value occupies multiple bytes, the marshaller performs byte shifting to stream their corresponding raw bytes into the network formatting (Big Endian). On the other hand, for the string, due to its variable length, our string protocol includes a length (4-byte int) payload prepending the actual string bytes.

3.2. Message Structure

Our message structure uses a custom binary protocol for communication between clients and servers. **The request message** sent by the client consists of **request ID (4 bytes), service ID (4 bytes), and the payload (bytes vary)**. Request ID is just for an identification number for each client request and service ID holds the information on which service to carry out. The payload refers to the set of information or arguments needed for the services.

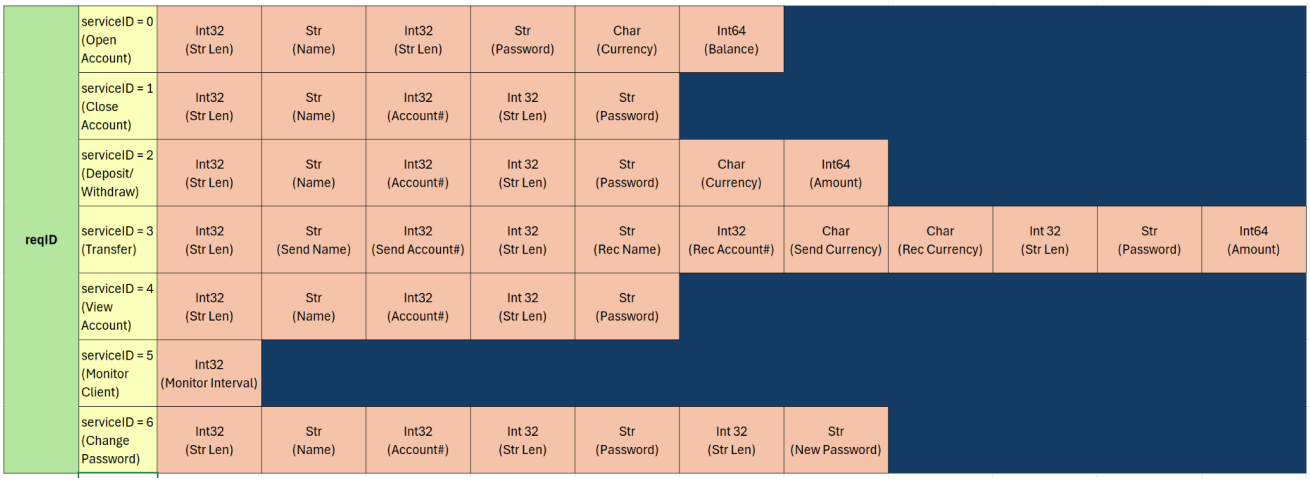


Fig 4. Request Structure

The reply message structure follows the similar structure as the request, but instead of service ID, it gives the status code to inform clients of the result which consists of **response ID (4 bytes), status code (1 byte), and the payload (bytes vary)** . The response ID MUST be the same as the current request ID processed. For successful operation, the output follows the output schema defined in the ***ServiceSchema*** while any error messages will contain a single string payload to inform clients about the error verdict. For monitoring services, it follows the exact same request-reply format.

responseID	Status Code	Int32 (Account#)					
	Status Code	Int64 (New Balance)					
	Status Code	Int 32 (Str Len)	Str (Rec Name)	Int32 (Rec Account#)	Char (Rec Currency)	Float (Exchange Rate)	Int64 (Remaining Balance)
	Status Code	Int32 (Account#)	Int32 (Str Len)	Char (Currency)	Int64 (Balance)		

Fig 5. Response Structure

4. Services

4.1. Service 1: Open New Account

This service allows new users to create a new account. The user will need to provide information such as username, password, currency, and initial balance. There are only 3 supported currencies: SGD, USD, and EUR. Once the account is successfully created, the server will generate and return the account number to the client.

4.2. Service 2: Close Existing Account

This service allows an existing account to be deleted. All the users need to provide are username, password, and account number. The server then will verify whether the indicated username exists or not and the indicated password and account number are correct or not, if yes then it will close that account and return a success to the client. Else, the server will return an error message to the client.

4.3. Service 3: Deposit/Withdraw Money

This service allows the client to either deposit or withdraw money from their bank account. It even supports currency exchange, meaning you can deposit or withdraw any currency. The server then will send the client the user's new balance.

4.4. Service 4: Transfer Money

This service allows an account to transfer some amount of money to another bank account. The user is required to input both sender and recipient details, each consisting of a username, account number, and currency, and also the sender's password and the amount of transfer. This service also allows users to transfer to another account with a different currency with a fixed predefined exchange rate for simplicity. If successful, the server will return the recipient name, recipient account number, recipient currency, exchange rate, and updated balance.

4.5. Service 5: View Account

This service allows users to view their account credentials by providing their username, password, and account number. The server will then return all the details of that dedicated account such as account balance and currency.

4.6. Service 6: Monitor Update

For some period of time (max. 30 minutes), the user can monitor all the client activities, including opening and closing accounts, depositing or withdrawing money, transferring funds, and changing passwords.

4.7. Service 7: Change Password

This service allows users to change their password, all the user needs to provide is username, account number, password, and new password.

5. Fault Tolerance

Fault tolerance is implemented to ensure that the system continues to function correctly even when failures occur. These failures may arise from network instability (e.g., packet loss or delays), duplicate or lost requests, or unexpected or invalid user inputs. Without proper fault tolerance mechanisms, such issues could lead to incorrect operations or even server crashes.

5.1. Timeout Retransmission

To handle unreliable communication (especially with UDP), the client uses a timeout mechanism. If a response is not received within a specified time, the request is retransmitted. This ensures that temporary network issues do not result in permanent request failure. However, retransmission introduces the risk of duplicate requests, which is why the duplicate request filter is implemented, specifically for the at-most-once semantics.

5.2. Filter Duplicate Requests

Since retransmissions may cause the same request to be processed multiple times, the server implements duplicate request filtering using a unique hashmap key constructed from the (client IP, client port, requestID) triplets. The server then uses an unordered *map* with this key to cache previously processed requests and their responses. If a duplicate request is received, the server returns the cached response instead of executing the operation again, ensuring **at-most-once semantics**.

```
struct MapKey {
    int32_t reqID;
    uint64_t networkaddr;
    bool operator==(const MapKey& other) const {
        return reqID == other.reqID &&
            networkaddr == other.networkaddr;
    }
};
```

```
struct MapKeyHasher {
    std::size_t operator()(const MapKey& k) const {
        return std::hash<int32_t>()(k.reqID) ^
            (std::hash<uint64_t>()(k.networkaddr) << 1);
    }
};
```

Fig 6. Map Key Implementation for Response Cache

6. Invocation Semantics

6.1. At-Least-Once

At-Least-Once semantics request retransmission but do not have duplicate filtering. In our implementation, the duplicate filtering is deactivated, so the server will blindly re-execute any duplicate messages that it receives, which can corrupt the application data in the non-idempotent operations.

6.2. At-Most-Once

At-Most-Once semantics request transmission and also filter duplicates. The server takes the client address (IP + port) and requests ID together as the key for duplicate checking. If the same request arrives again from the same client address, the server will not execute the service again and returns the cached response instead. This is the safest approach to avoid data corruption by making sure that any request is STRICTLY processed once in the runtime, though it may incur additional memory expenses to hold the response cache.

6.3. Testing

6.3.1. Idempotent

We use View Account Balance to showcase the example of idempotent operation. Here, under the at-least-once semantic invocation server configuration, we can see that the loss response packet triggers the client to resend another request packet to the server. As we can see that both attempts trigger the server state “PROCESSING SERVICE ID 5”, indicating that the server does not know whether the packet is a duplicate and it is not aware of the existence of cache. Despite this, the response packet remains the same, demonstrating that idempotent operation is not affected by the reprocessing in the at-least-once semantics, hence it will not be affected by the at-most-once semantics as well which does not do any reprocessing for duplicate messages.

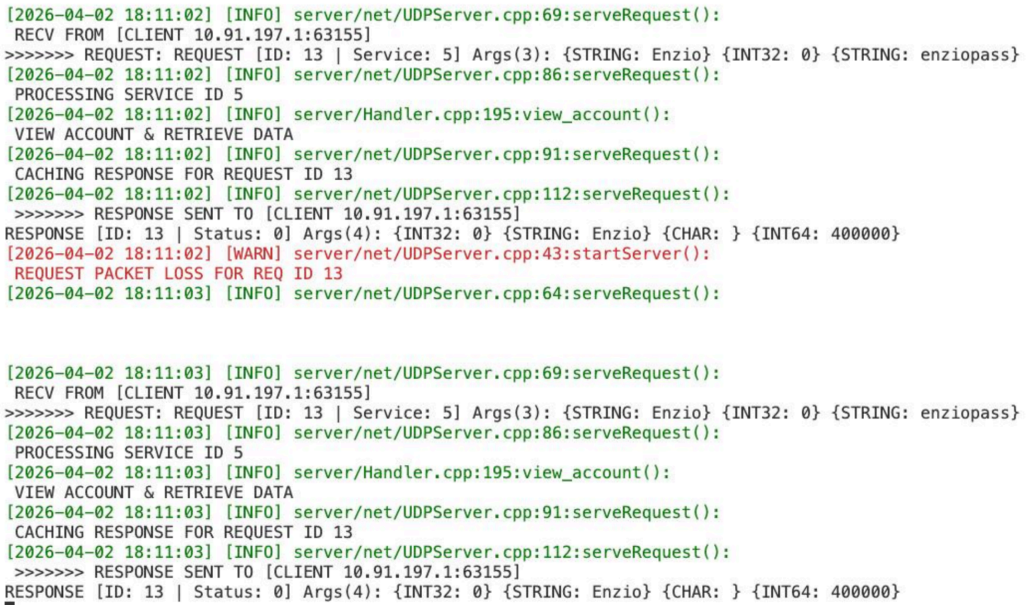


Fig 7. At-least-once semantics on View Account Balance Operation (Idempotent)

6.3.2. Non-Idempotent

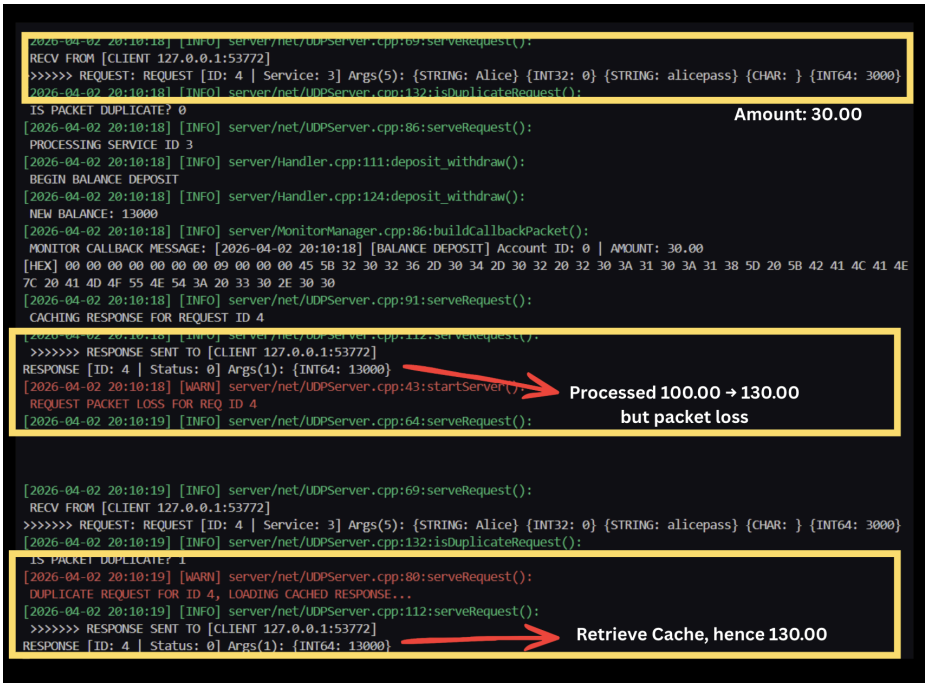


Fig 8. At-Most-Once Semantic Invocation on Deposit/Withdraw Operation (Non-Idempotent)

As can be seen here, the duplicate filtering successfully detects the previously processed duplicate messages and proceeds to return the cached result, so the output of the program remains to be $130.00.

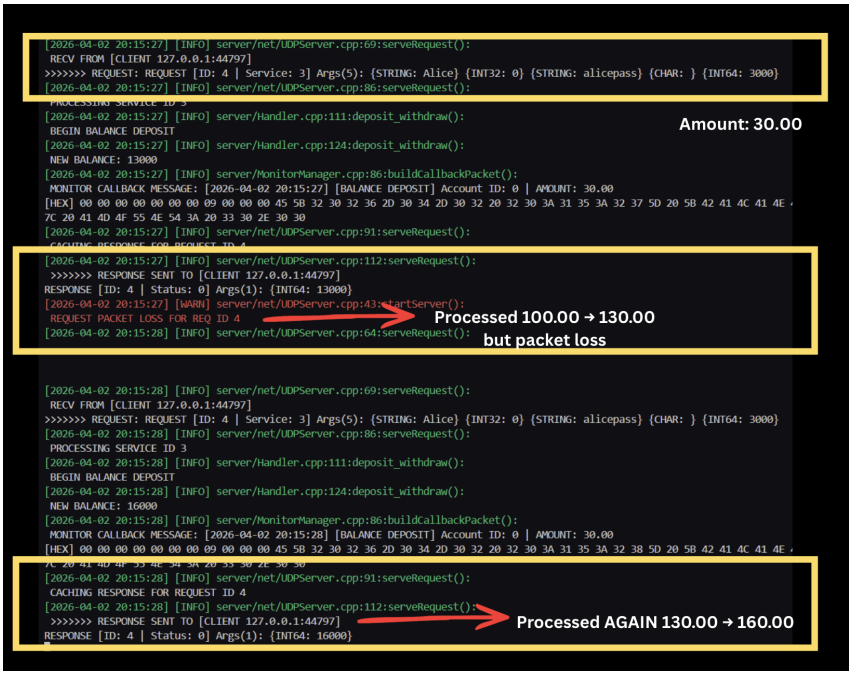


Fig 9. At-Least-Once Semantic Invocation on Deposit/Withdraw Operation (Non-Idempotent)

However without any duplicate filtering as shown in the at-least-once semantics, the request is further reprocessed again causing the balance to be added twice, causing unwanted data corruption.

7. Conclusion

In this project, we designed and implemented a distributed banking system using a client–server architecture with UDP as the communication protocol. Despite UDP's connectionless and unreliable nature, the system was able to handle client requests effectively through structured message handling, marshalling, and response management.

To ensure correctness and reliability, we incorporated appropriate invocation semantics and mechanisms such as request identification and response handling to manage duplicate or lost messages. These features allowed the system to maintain consistent behavior and support core banking functionalities, including account operations and monitoring services. Overall, the project demonstrates a functional and reliable distributed system that effectively operates over an unreliable communication protocol while meeting the required service specifications.